



运算器

定点加减法

定点数的加减运算

采用补码，运算规则：

- 符号作为数的一部分直接参与运算
- 操作为加，则直接相加
- 操作为减，则将减数变补后再与被减数相加

溢出判别

1. $\overline{F_a F_b} F_s + F_a F_b \overline{F_s} = 1$
 - F_a, F_b : 加数，被加数符号位
 - F_s : 运算和符号位
2. $C_f \oplus C = 1$
 - C_f : 符号位运算产生的进位，即 $F_a F_b$
 - C : 最高有效位产生的进位，即 F_s
 - $C_f \oplus C = F_a F_b \oplus F_s$
3. 使用双符号位
 - 在最高位补符号位 ($0 \rightarrow 00, 1 \rightarrow 11$)
 - 计算结果双符号位不同代表溢出

定点原码一位乘法

- 数值为绝对值相乘
- 符号为两数符号异或
- 运算规则：
 - 结果置0，根据乘数最末一位判断
 - 最末位是1，结果加被乘数后右移一位
 - 最末位是0，结果直接右移一位
 - 如乘数为n位，则需要进行n次累加移位

定点补码一位乘法

设 $[x]_{\text{补}} = x_0.x_1x_2 \dots x_n, [y]_{\text{补}} = y_0.y_1y_2 \dots y_n$, 将符号位参与计算, 运算数以补码表示且被乘数与部分积都取双符号位。

Booth算法的基本流程：
符号位参与计算，运算数以补码表示且被乘数与部分积都取双符号位。
记部分积初值为0，乘数末位增加附加位，初值置0
根据 (y_n, y_{n+1}) 的取值，按下表循环执行n+1次，得到了n次对部分积的累加，并且最后一次部分积不右移，此时执行了n次右移

Yn	Yn+1	操作
0	0	+0, 右移一位
0	1	+ $[x]_{\text{补}}$, 右移一位
1	0	+ $[-x]_{\text{补}}$, 右移一位
1	1	+0, 右移一位

定点原码一位除法

加减交替算法：

- 被除数先减除数，结果（余数）左移一位
- 结果（余数）为正商上1；结果（余数）为负商上0
- 如上次商为1时，则被除数减除数；如上次商为0时，则被除数加除数

定点补码一位除法

加减交替算法：

- 如被除数与除数同号，先减除数
- 如被除数与除数异号，先加除数
- 余数和除数同号，商上1，余数左移一位，下次减除数
- 余数和除数异号，商上0，余数左移一位，下次加除数

原码加减交替法和补码加减交替法对比：

除 法 类 型	符号位参与运算	加 减 次 数	移 位		上商、加减原则	说 明
			方 向	次 数		
原码加减交替法	否	N+1或N+2	左	N	余数的正负	若最终余数为负，需恢复余数
补码加减交替法	是	N+1	左	N	余数和除数是否同号	商末位恒置1

浮点运算

浮点数的加减法

- 对阶
 - 阶码大的数不变
 - 阶码小的数的尾数右移并相应增大阶码，直到阶码相等
 - 原码表示：右移补0
 - 补码表示：右移补符号位
- 运算
 - 尾数进行相加减
 - 采用双符号位
- 规格化
 - 两个符号位不同，说明溢出，右规
 - 尾数右移一位
 - 阶码+1
 - 两个符号位相同，但最高数值位与符号位相同，左规
 - 连续左移，直到最高数值位与符号位不同
 - 阶码减去相应的值
- 舍入
 - 右规或对阶时，需要保留若干位尾数
 - 常采用0舍1入进行保留
- 检查阶码是否溢出
 - 下溢：置结果为0
 - 上溢：置溢出标志

浮点数的乘除法

结果的阶码为运算两数阶码之和（相乘）/差（相除），尾数为运算两数尾数相乘/除

阶码运算

- 若阶码用补码表示，则采用前述定点补码加减法规则和判定溢出的方法
- 若阶码用移码表示

$$\begin{aligned}[X + Y]_{\text{移}} &= X + 2^n + Y = [X]_{\text{移}} + [Y]_{\text{补}} \\ [X - Y]_{\text{移}} &= X + 2^n - Y = [X]_{\text{移}} + [-Y]_{\text{补}}\end{aligned}$$

即加/减数符号位取反后相加/减

- （阶码运算）采用双符号位，最高位置0，运算结果最高位为1则说明溢出

乘法运算步骤

1. 检测两数是否为0
2. 两数阶码相加
3. 两数尾数相乘
4. 尾数规格化
5. 判断阶码是否溢出

除法运算步骤

1. 检测两数是否为0
2. 尾数调整
 - 检测被除数尾数绝对值是否小于除数尾数绝对值
 - 若不是，被除数尾数右移一位，阶码+1
3. 被除数阶码减除数阶码
4. 被除数尾数除以除数尾数
5. 尾数规格化
6. 判断阶码是否溢出

BCD码运算

加法

按二进制相加，对结果进行修正

- 结果某位和大于9，则该位+6并进位
- 运算时低位向高位进位时，低位+6

减法

按二进制相减，对结果进行修正

- 运算时低位向高位借位时，低位-6